

Serendip: A Multi-Agent Travel Planner

DASC7606C Group Project Proposal – Track 2: Agentic Framework Design

Group 7

28 September 2026

1 Track, topic and objectives

Track: Track 2, Agentic Framework Design.

Topic: Serendip, a multi-agent planner that turns a loose travel request, typed as one sentence (“I want to spend five days in Hangzhou”) or entered through a short form, into two contrasting day-by-day itineraries built from live data. One plan covers the classic highlights; the other goes off the beaten path. Users can then edit any part of a plan by selecting blocks and describing the change. Code: <https://github.com/y1112002/Travel-Agent>.

Problem. Planning even a short domestic trip means checking many separate sources: trains or flights, hotels, the weather, attractions, local events, food and promotions, and then fitting them into a schedule that suits the traveler’s budget, pace and interests. Travel platforms sell the pieces but leave the assembly to the user; social platforms offer inspiration without live prices or weather; general chatbots write itineraries without live data and can invent venues. Our users are students and young professionals planning short trips within mainland China who want a usable plan in minutes rather than an evening of research.

Objectives

1. Turn a loose request into two contrasting, day-by-day plans grounded in live data on weather, hotels, transport, attractions, events, food and promotions.
2. Keep plans grounded: at least 90% of named attractions, hotels and restaurants appear in the search results, with booking or information links attached.
3. Cut end-to-end latency from the current ~91 seconds to a median of 60 seconds or less.
4. Let users edit selected blocks with a plain-language instruction, without regenerating the whole plan.
5. Add a trained component: a small open model fine-tuned with LoRA for request parsing, compared with the current API model.
6. Extension, if time allows: compare 2–3 candidate destinations or a flexible date window and recommend one.

How this plan covers Track 2. Our contribution goes beyond a single model or API call: it is a complete workflow from a loose request to two checked, editable plans. A working end-to-end version already runs.

Track 2 requirement	Where we address it	Points
Problem and users	Short-trip planners in mainland China; many sources to combine by hand (Section 1)	15
Tool design and usability	Profile, trip form or one-sentence input, two plans, block editing, trip memory (Section 3)	15

Track 2 requirement	Where we address it	Points
Deep learning approach	LLM agents with structured output, LLM feature extraction, LoRA fine-tuned parser, embedding-based matching (Section 3)	20
Implementation and demo	Working React + FastAPI + LangGraph system; live demo and backup recording (Sections 3, 4)	25
Evaluation and limitations	Test requests, baselines, ablations, user study, latency, failure analysis, risks (Section 3)	15
Report, presentation and Q&A	5–8 page report, 10-minute talk, every member prepared for Q&A (Section 4)	10

2 Member roles and responsibilities

Each of the 10 members owns one module end to end and one section of the final report; everyone prepares to answer Q&A on their own module.

#	Member	Role	Responsibilities	Report section
1	Dai Enhe	Product manager	Scope and priorities, daily stand-up, profile and trip form content, demo script, user study, report editing	Problem, users, design
2	Ying Le	Tech lead	LangGraph Orchestrator, JSON contracts between agents, FastAPI back end and <code>/api/plan</code> pipeline, integration, latency work	Architecture
3	Zhang Yuetao	SearchAgent: transport and deals	Flight and train search (round trip), promotions, via the Fliggy FlyAI tools	Data sources
4	Chen Xi	SearchAgent: destination content	Weather, hotels, attractions, events and food; feature extraction; search cache and test snapshots	Data; deep learning
5	Zhang Zhipeng	QuestionnaireAgent and preference matching	Preference questions from profile and search results; embedding-based matching of interests to attractions and events	Tool design; deep learning
6	Yan Kaifeng	PlanAgent	Two-style plan generation, block format, block-level editing	Tool design
7	Zhao Jiaqi	ValidateAgent, safety and privacy	Review rules and feedback loop, rule-based grounding checks, phone-number hashing, data deletion, prompt-injection guard	Risks and limitations
8	Xu Xushang	Model fine-tuning	Parsing dataset, LoRA fine-tuning of a small open model, comparison with the API model	Deep learning
9	Qiu Leheng	Front end	React pages for login, profile, trip form, planning and block editing; progress display	Tool design
10	TBD	Evaluation lead	Test requests, baselines and ablations, metrics scripts, user-study analysis, backup recording	Evaluation

3 Planned approach

User workflow

1. The user logs in with a phone number and a verification code (development mode; no real SMS).
2. On first use, the user fills a profile: age group, gender, identity, home city and travel styles (e.g., culture, food, nature, family, photography).
3. For each trip, the user fills a short form (destination, origin, dates, travelers, budget tier and amount, purposes) or types one sentence, which is parsed into the same fields.
4. SearchAgent gathers live data; PlanAgent writes two plans, “Classic highlights” and “Off the beaten path”; ValidateAgent reviews them.
5. The user compares the plans, selects blocks and types an instruction (e.g., “a cheaper hotel”); only those blocks are rewritten. The chosen trip is saved as a trip memory, and user actions are logged as behavior signals.

Advantages over existing tools. Travel platforms such as Fliggy and Trip.com list flights, hotels and tickets but leave the itinerary to the user; Xiaohongshu-style platforms are unstructured and not aware of live prices or weather; general chatbots produce itineraries without live data and can invent venues. Serendip combines live data from several sources, offers two contrasting plans, checks them before showing them, and supports editing at the level of single blocks. The final report will cite official sources for these comparisons.

Architecture

A React 18 + TypeScript front end, a FastAPI + SQLAlchemy back end on SQLite, and five agents. Each agent is a separate module that reads and writes JSON; the Orchestrator connects them with a LangGraph state graph (search → plan → validate, with a conditional loop back to plan).

- **Orchestrator** loads the stored profile, runs the graph and saves the result as a trip memory.
- **SearchAgent** parses one-sentence requests and calls 6–8 tools in parallel: weather (Open-Meteo), hotels, attractions, promotions, flights and trains (Fliggy FlyAI), and events and food (Tavily web search). Results are cached. The same tools are also exposed as an MCP server.
- **QuestionnaireAgent** writes 4–6 multiple-choice questions from the profile and search results. It is built but not yet in the default graph; we will add it as an optional step and measure its effect.
- **PlanAgent** writes the two plans in parallel from the profile, trip details and search results, turns them into blocks (time, type, name, note, link), and rewrites only the selected blocks when the user edits.
- **ValidateAgent** checks each plan for budget, transport, preference, weather, timing and completeness problems, and returns issues with feedback for PlanAgent.

Current status. The full workflow runs end to end. Latency fell from nearly 10 minutes to about 91 seconds (parsing ~11 s, search ~22 s, planning ~40 s, validation ~15 s) through parallel tool calls, parallel plan generation, shorter prompts and a single review round. Next steps: merge feature extraction into one call, parse simple requests with rules first, stream progress to the page, and add in-city transport times.

Deep learning components

- **LLM agents with structured output.** DeepSeek models, through an OpenAI-compatible API, parse requests, extract features, write plans, review plans and generate questions, all as JSON. We chose a fast model after measuring that reasoning tokens dominated latency and caused truncated JSON.
- **LLM feature extraction.** Long attraction, event and food descriptions are reduced to short feature tags, which cut the planning prompt from about 25,000 to 11,700 characters.
- **Trained component (planned).** Parsing a request takes about 11 seconds with the API model. We will fine-tune a small open model (e.g., Qwen, 1.5B–4B parameters) with LoRA on about 800

synthetic Chinese trip requests paired with gold JSON, test it on about 100 requests written by classmates, and compare field accuracy, latency and cost with the API model.

- **Embedding-based matching (planned).** Multilingual sentence embeddings (e.g., BGE-M3) will score attractions and events against the profile’s travel styles, so PlanAgent receives a shorter, better-ranked candidate list.

Data. Live data from Fliggy FlyAI (hotels, flights, trains, attractions, promotions), Tavily (events, food), and Open-Meteo (weather). For a reproducible demo and evaluation, we will freeze the cached search results for every test request as a snapshot.

Compute. LoRA fine-tuning and embedding inference will run on the HKU GPU Farm (RTX 4080). As an optional goal, we will test the fine-tuned model on Moore Threads GPUs and report profiling and benchmark results.

Evaluation

About 30 test requests (typical, edge-case, and vague or mixed Chinese–English) run on the frozen snapshot, plus a user study with 8–10 classmates.

Metric	Target
Plans returned as valid JSON with every day filled	$\geq 95\%$
Named attractions, hotels and restaurants found in the search results	$\geq 90\%$
Plans within the stated budget	$\geq 90\%$
Attraction overlap between the two plans (lower means more contrast)	Report
Block edits that change only the selected blocks and follow the instruction	$\geq 85\%$
Parsing field accuracy, LoRA model vs. API model	Report
Latency from request to both plans (median)	≤ 60 s
User study: usefulness rating (1–5) and preferred plan style	Report

Baseline: a single LLM prompt without tools. Ablations: the full system without ValidateAgent, with and without QuestionnaireAgent, and without feature extraction. We will report where the system fails, with worked examples.

Limitations and risks

Risk or limitation	Mitigation
Live data varies; free-tier Fliggy prices may be blurred; web results change	Frozen snapshots for demo and evaluation; prices shown with source links
The LLM names a venue or price that is not in the data	Names checked against search results; links filled in from the data; ValidateAgent review
Latency of about 91 seconds	Parallel calls, caching, planned speed-ups, visible progress
Phone numbers currently stored in plain text and used as user IDs	Test numbers only in the demo; hash identifiers; add a data-deletion endpoint
Prompt injection hidden in web search results	Tool output treated as data; read-only tools; no booking or payment
No in-city transport data yet	State it as a limitation; add a map routing API if time allows
Small test set and a student-only user study	State the limits and report results by request type

Timeline (two weeks)

Days	Milestone
1–2	Freeze JSON contracts and block format; write test requests; freeze search snapshots
3–6	Speed-ups, QuestionnaireAgent option, grounding checks, privacy fixes; build the LoRA dataset and train
7	Full demo run with profile and trip details on snapshot data
8–10	Evaluation, ablations, user study, LoRA comparison, fixes
11–13	Code freeze, backup recording, report, slides, rehearsals
14	Presentation

4 Expected deliverables

Deliverable	Content
Working demo	Web app covering login, profile, trip input, agent progress, two plans and block editing; backup screen recording of the same path
Source code	The repository above: front end, back end, Orchestrator and five agents, with setup instructions, sample inputs and test snapshots; no API keys
Report (5–8 pages)	Problem and users, tool design, deep learning approach, implementation and evaluation results, limitations and risks
Presentation (10 min + 5 min Q&A)	Problem, workflow, architecture, live demo, results, limitations
Contribution statement	Each member’s work, following the roles in Section 2

Contribution evidence. Individual scores apply a contribution factor to the group score, so each member keeps a record of their work: commits, task-board items and short notes.

5 LLM usage statement

- **Whole proposal:** We used Claude (Anthropic) to summarize the implemented system from the group’s code repository. The group reviews and edits the final wording before submission.

We will update this statement in the final report to cover LLM use in code, analysis and writing.